Computers & Security 150 (2025) 104298



Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose





Unmasking the hidden credential leaks in password managers and VPN clients

Efstratios Chatzoglou [a], Vyron Kampourakis [b], Zisis Tsiatsikas [a], Georgios Karopoulos [c,*], Georgios Kambourakis [a]

[a] University of the Aegean, Karlovasi, 83200, Samos, Greece
[b] Norwegian University of Science and Technology, Gjøvik, 2802, Norway
[c] European Commission, Joint Research Centre (JRC), 21027, Ispra, Italy

ARTICLE INFO

Keywords:
Password managers
VPN clients
Security
Data leaks
Vulnerability

ABSTRACT

With the rapid growth of software services and applications, the need to secure digital assets became paramount. The introduction of Password Manager (PM) and Virtual Private Network (VPN) software was established as a prerequisite toolkit to bolster the end-user arsenal. As a matter of fact, these types of artifacts have been around for at least 25 years in various flavors, including desktop and browser-based applications. This work assesses the ability of 12 desktop PM applications, 5 browsers with integrated PM, and 12 PMs in the form of browser plugins, along with 21 VPN client applications, to effectively protect the confidentiality of secret credentials. Our analysis focuses on the period during which an app is loaded into RAM. Despite the sensitive nature of these applications, our results show that across all scenarios the majority of PM applications store plaintext passwords in the system memory; more specifically, 75% (or 9 out of 12) of desktop PM applications, 100% (5 out of 5) of browser PMs and 75% (or 9 out of 12) of PM browser plugins leak such sensitive information. In addition, 33% (or 7 out of 21) of VPN applications leak user credentials. This practice of storing cleartext sensitive information in system memory is widely recognized as a weakness, having also been registered as CWE-316. At the time of writing, merely four vendors have recognized our exploits as vulnerabilities. Three of these vendors have assigned the relevant Common Vulnerabilities and Exposures (CVE) IDs, namely CVE-2023-23349, CVE-2024-9203, and CVE-2024-50570, whereas the fourth one will issue a CVE ID once it implements the relevant fixes. The remaining vendors have either chosen to disregard or downplay the severity of this issue.

1. Introduction

Despite Bill Gates' prediction of the death of passwords twenty years ago (Kotadia, 2004), we are still using them today. In addition, there have been major data breaches due to weak or insecure passwords, such as in the recent cases of SolarWinds (Fung and Sands, 2021) and Colonial Pipeline (Turton and Mehrotra, 2021). Passwords come in different forms, such as Personal Identification Numbers (PINs) and passphrases, and frequently as one of the components of multifactor authentication. Between the impractical solution of remembering all passwords and the insecure one of using the same password everywhere, one of the most popular ways for protecting credentials is Password Managers (PMs).

Apart from PMs for safeguarding and conveniently managing passwords, modern enterprises rely on other security-critical tools to protect remote access and secure communications, such as Virtual Private Network (VPN) clients. These tools, like PMs, handle authentication credentials, which are temporarily stored in memory during operation. Any exposed credentials obtained from a victim's machine memory could have severe security implications for the other end of the thread, *i.e.*, the enterprise's internal assets. Once an evildoer gains unauthorized access using compromised credentials, they can move laterally toward high-value assets (Smiliotopoulos et al., 2024). A single memory leak could enable attackers to infiltrate the entire corporate network, leading to potentially devastating consequences, like data theft or ransomware attacks, as blatantly confirmed by the SolarWinds (Fung and Sands, 2021) case. Given the widespread adoption of password managers and other security tools that handle sensitive credentials, a key question is: How secure is this sensitive data stored in memory by these applications, and to what extent are they vulnerable to memory dumps or extraction attacks?

Looking first at how software developers and vendors handle security throughout the entire software development lifecycle, including

* Corresponding author.
E-mail address: georgios.karopoulos@ec.europa.eu (G. Karopoulos).

https://doi.org/10.1016/j.cose.2024.104298
Received 1 November 2024; Received in revised form 16 December 2024; Accepted 19 December 2024
Available online 28 December 2024
0167-4048/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

requirements gathering, design, development, testing, deployment, and maintenance, PMs and VPN applications capitalize on relevant principles, methodologies, and tools, as well as standards and best practices for secure software development. While these practices can significantly enhance security, absolute security cannot be guaranteed. Risk management is a crucial component of a secure software development lifecycle. During this process, software vendors need to carefully evaluate and prioritize security risks, considering factors such as likelihood, impact, and cost of remediation. However, in some cases, vendors may choose to ignore or delay addressing security issues that they deem less critical or more expensive to fix. Interestingly, in our study, we contacted several PM and VPN vendors to report security issues related to memory leaks and extraction attacks. Until now, only four of these vendors have acknowledged the issues and taken steps to address them.

Focusing on PMs, both (Pearman et al., 2019) and Fagan et al. (2017) found that PM users often prioritize convenience over security. While (Pearman et al., 2019) concentrated on standalone PMs, (Fagan et al., 2017) compared PM users to non-users. These findings suggest that PM users do not always prioritize security when adopting such applications. This trade-off between convenience and security is likely to extend to tools like VPNs, where ease of access may come at the cost of secure memory management. While there may be exceptions, many users may be more willing to accept potential security risks in exchange for convenience and ease of use.

Despite frequent mentions of PMs and VPNs in the media in the form of advertisements, comparisons, or security analyses (Wired, 2024; Anon, 2024), no silver bullets exist regarding absolute security. Even popular choices, recognized for their security level, can suffer from security issues such as vulnerabilities, data breaches, or weak encryption (PASCU, 2017). For example, a recent critical vulnerability in KeePass allowed attackers to recover the master key safeguarding the PM's vault (sysdig, 2023). Given the potential security risks associated with even popular PMs and VPNs, our study aims to understand the level of private information they potentially leak. We focused on the period during which these applications are loaded into system memory and analyzed 12 desktop PMs, 5 browsers, and 12 browser plugin PMs, as well as 21 VPN clients. These applications were selected as representative cases of security controls used by many organizations to effectively comply with security practices and standards, such as ISO 27001 (International Organization for Standardization, 2022).

The principal contributions of this research are as follows:

- We examine which modern PMs and VPNs allow the extraction of plaintext credentials from system memory, offering exploits that can streamline the retrieval process, and investigate to which extent the leaking information reveals repetitive patterns that augment the attack surface.
- We assess PM and VPN vendor conformity to well-established standards and guidelines, such as the ISO/IEC 27001, NIST-SP 800, OWASP Top 10, and OWASP Secure Coding Practices, concerning the exposure of private information.
- Following responsible disclosure, we contacted the respective PM and VPN vendors privately, provided detailed information about the identified vulnerabilities, and briefly described our correspondence with them.

This paper is an extended and improved version of previous work presented at the 2024 ICT Systems Security and Privacy Protection (IFIP SEC) (Chatzoglou et al., 2024b). In this extension, we have improved our study on a four-axis basis:

- **Improvement #1:** We revisit the latest versions of the already scrutinized PMs to emphasize the enduring impact of our previous research. Namely, we demonstrate in which of the considered scenarios the PMs: (a) continue to show credential leakages, (b) have implemented corrective actions to address these leakages, and (c) reveal new leakages in scenarios where none was previously detected.
- **Improvement #2:** We extend our experimentation benchmark in browser-native PMs and VPN clients. In this way, we showcase that sensitive information leakage is not exclusive to PMs but rather affects a wide spectrum of commonly used applications, underscoring the pervasive nature of these vulnerabilities across diverse digital tools. In this context, we also contacted VPN vendors to share our findings, following ethical vulnerability disclosure.
- **Improvement #3:** We enhance our exploitation methodology by introducing a scheme that leverages the repetitive occurrences of plaintext credentials. The purpose of this annex is to shorten the pattern-based output of the text extracted from the original dump files to subsequently identify the desired credentials effortlessly.
- **Improvement #4:** We further explore pertinent standards and guidelines, including ISO/IEC 27001 (Anon, 2022a) and NIST-SP 800 (Grassi et al., 2020), to pinpoint specific security requirements and controls relevant to preventing information leakage. In the context of this effort, we also expand upon the previously proposed best practices by incorporating additional countermeasures.

Apart from the above-described major improvements we also enhanced our study in the following aspects. First, we expand our threat model by detailing the key characteristics, goals, and capabilities of potential attackers to better comprehend the nuances of credential leakage attacks and to assess the potential impact of various attack scenarios on user data security. Second, since some of the communication with the PM vendors was still ongoing at the time of publishing our previous study, this extended version now includes the latest findings that have emerged, such as CVE-2024-9203 and CVE-2024-50570. Third, we stretch the discussion around the conducted experiments providing more detailed information regarding the experimental setup, in-depth observations from the experiments, and comparisons between PMs and VPNs regarding the extent of leakage they permit. Last, we broaden the discussion of relevant works by (a) including similar works that deal with sensitive information leakage in VPN applications, and (b) conducting a comparison of our work with the rest of the relevant literature.

The rest of the paper is structured as follows. The related work is given in the next section. Section 3 presents the threat model. Section 4 details our methodology, the derived results, and ways of exploitation in the context of red teaming. Section 5 explains best practices, including possible mitigation strategies. The last section concludes and offers directions for future work.

2. Related work

This section discusses research regarding the security of PM and VPN applications, reviewing literature published over the last ten years. Table 1 summarizes the main characteristics of each related study, concentrating on password leaks in memory which is the focus of our paper.

The first set of related works focuses on PM applications that expose sensitive data while running on different operating systems, namely Windows, Linux, and Android. The authors in Apostolopoulos et al. (2013) investigated the potential leakage of private data from 30 Android applications of different types residing in RAM. According to the findings, all 17 PM applications examined were found to disclose information that facilitated the retrieval of user passwords. A similar research in Lee et al. (2019) presented an analysis of Android applications, including 4 PMs, on the persistence of passwords in system memory even after they are not needed, finding that all tested apps are vulnerable. Based on their results, one of the main components responsible for leaking private information is UI widgets. To address this, they proposed an enhancement to the Android platform by introducing SecureTextView, a secure version of Android TextView, which employs the zeroization of specific buffers. In a related research (Malliaros et al.,

Table 1
Related work on password manager and VPN applications, sorted by year, showing works that examine password leaks in system memory, and assigned CVEs and exploits related to these password leaks. (†: browsers, *: one CVE ID already issued and the other will be issued once the vendor implements the relevant fixes, **: only a subset of applications was studied).

Work	Year▾	Platform	# of apps	Password leaks	CVEs	Exploits
Password managers						
This work	2024	Desktop, browser	24	✓	2	23
Gangwal et al. (2023)	2023	Android	7	✗	–	–
Sabev and Petrov (2022)	2022	Android	2	✗	–	–
Oesch and Ruoti (2020)	2020	Desktop, browser	13	✗	–	–
Carr and Shahandashti (2020)	2020	Desktop, browser, Android	5	✓	–	–
Barten (2019)	2019	Browser	1	✓	–	–
Lee et al. (2019)	2019	Android	4	✓	–	–
Luevanos et al. (2017)	2017	Desktop, browser, iOS, Android	3	✓	–	–
Malliaros et al. (2016)	2016	Desktop	4†	✓	–	–
Gray et al. (2016)	2016	Desktop	3	✓	–	–
Li et al. (2014)	2014	Browser	5	✗	–	–
Silver et al. (2014)	2014	Desktop, browser, iOS, Android	10	✗	–	–
Apostolopoulos et al. (2013)	2013	Android	17	✓	–	–
Zhao et al. (2013)	2013	Browser	2	✗	–	–
VPN						
This work	2024	Desktop	21	✓	2*	2
Abbas et al. (2023)	2023	Desktop	4	✗	–	–
Wu et al. (2023)	2023	Desktop, iOS, Android	140	✗	–	–
Ramesh et al. (2022)	2022	Desktop	80	✗	–	–
Wilson et al. (2020)	2020	iOS	57	✗	–	–
Bui et al. (2019)	2019	Desktop	30	✓**	–	–
Zhang et al. (2018)	2018	Android	84	✗	–	–
Ikram et al. (2016)	2016	Android	283	✗	–	–
Perta et al. (2015)	2015	Desktop, iOS, Android	14	✗	–	–

2016), the authors explored zeroization techniques across MS Windows and Linux environments. Although this work does not focus on PMs, it is considered relevant since it studies credential extraction from memory allocated to browsers. The authors concluded that only the MS Windows operating system provides a reliable function that can be used for this purpose.

Extending the analysis to web-based PMs that offer standalone applications and/or browser extensions, the authors of Oesch and Ruoti (2020) conducted a security evaluation of thirteen web-based PMs. Their evaluation covered both the standalone application and browser extension-based versions, encompassing the entire PM lifecycle, from password generation and storage to autofill functions. Despite the improvement of the scrutinized PMs in terms of security compared to prior evaluations, persistent issues remain. These include unencrypted metadata, insecure defaults, and vulnerabilities to clickjacking attacks. The authors explored the methods that each of the PMs utilizes to securely store client data in the respective database files. However, they overlooked how the very same data are treated by the host's OS and how they are stored in RAM.

In the same vein, the authors in Li et al. (2014) performed a security analysis of five web-based PMs, uncovering varying susceptibilities across all the examined PMs. Among the identified vulnerabilities were issues related to logic and authorization, misinterpretation of the web security model, and sensitivity to CSRF/XSS attacks. Nevertheless, the authors specifically focused on vulnerabilities stemming from the web-based nature of PMs. In contrast, our study takes a different angle by delving into the local security posture of PMs, with a particular emphasis on examining how sensitive data are stored within the users' RAM.

Another study (Barten, 2019) presented flaws in the LastPass browser extension. In this work, the authors performed a password extraction attack directly from the extension's memory while the extension was active. Similarly, the work in Zhao et al. (2013) provided a vulnerability analysis of LastPass and Roboform, which both provide browser extensions to run the software and store passwords in the cloud. They identified potential threats, including storing credentials in plaintext locally and/or on cloud servers. In contrast to our work, they only study the recovery of plaintext passwords from the local PM database and not from memory.

A second set of works focused on data leakage during the autofill function that modern PMs offer. The authors of Silver et al. (2014) studied how different PMs handle the automatic password filling. Their work encompassed three categories of password managers: (a) built into browsers, (b) used in mobile devices, and (c) third-party applications. Their findings suggested that the various autofill policies followed by these tools can expose users to serious risks, such as a remote attacker stealing multiple passwords from their PM. Similarly, (Gangwal et al., 2023) presented a technique, called AutoSpill, to leak users' saved credentials during an autofill operation on a webpage loaded into an application's WebView.

As a third tendency, some authors have followed forensic techniques to study the security of PM applications. In Gray et al. (2016), three desktop PMs were analyzed, namely KeePass, Password Safe, and RoboForm, following a systematic forensic analysis approach. The findings of this study showed that unencrypted passwords and sensitive data were found in the clipboard, Temp folders, Page files, or the Recycle Bin. In addition, the authors of Sabev and Petrov (2022) followed a digital forensic procedure to analyze the storage security of two Android PM applications. However, their focus is on protecting long-term data stored in Android devices, leaving out attacks targeting encryption keys in main system memory.

In a similar direction to our work, the study in Luevanos et al. (2017) scrutinized the security of three open-source PMs against various types of attacks, both new and already known. The PMs they studied were Passbolt, Padlock, and Encryptr; among other vulnerabilities, the authors found that all three applications were susceptible to keyloggers and clipboard attacks. Furthermore, the authors of Carr and Shahandashti (2020) analyzed 5 popular PMs against already disclosed and newly discovered vulnerabilities. In particular, their findings indicated that 4 of the 5 PMs studied, namely Dashlane, LastPass, Keeper, and RoboForm, were prone to clipboard attacks.

When it comes to tools for password recovery from PMs, *LaZagne* (Anon, 2024b) is an open-source application capable of recovering locally stored computer passwords. It supports various methods that applications use to store their passwords, such as plaintext, APIs, custom algorithms, and databases. Among the supported applications are popular browsers, mail clients, chat apps, databases, and memory dumps

from password managers such as KeePass. *Pandora* (Anon, 2024f), the tool we have developed for our experiments, focuses on PMs and delves deeper than LaZagne. It uncovers passwords that are inadequately managed by 18 different PM implementations, spanning Windows apps, browsers, and browser plugins. Unlike *LaZagne*, which typically requires administrative privileges, *Pandora* in most cases operates with standard user access.

To date, several studies have examined security and privacy concerns associated with VPNs within mobile ecosystems. More specifically, the authors in Wilson et al. (2020) examined the security level of 57 iOS VPN applications. According to their results, most applications suffered from weaknesses, such as HTTP usage, DNS leakage, and failure to adhere to best practices when using IPsec. Similarly, the authors in Zhang et al. (2018), analyzed 84 popular Android OpenVPN apps for security issues. Their findings revealed various problems related to customized insecure protocols, weak client authentication, and incorrect file permissions on Android. These issues mainly stem from insecure modifications and configuration of the OpenVPN protocol, which can lead to attacks such as man-in-the-middle (MitM) and traffic decryption. In Ikram et al. (2016), the authors perform static and dynamic analysis of 283 apps that use Android VPN permission. Their findings include serious security issues, such as insecure VPN tunneling protocols, IPv6 and DNS traffic leakage, apps performing TLS traffic interception, and the inclusion of malware in VPN apps.

Focusing on the desktop landscape, the study in Bui et al. (2019) examined thoroughly the 30 most popular VPN clients for vulnerabilities. In most cases, the root problem was related to bad configuration which allowed the attacker to launch MitM attacks. Moreover, it was possible to steal user credentials using impersonation techniques. Similarly to our work but limited to the study of the OpenVPN protocol only, the authors found that some VPN applications leak user credentials by storing them in local configuration files accessible to all system users. The authors in Perta et al. (2015), researched the security level of 14 popular VPN services. According to their results, the vast majority exhibit IPv6 traffic leakage. Moreover, using advanced DNS hijacking techniques, they managed to intercept all traffic on the network. The work in Wu et al. (2023) assessed the security level of academic VPN solutions by analyzing: (a) VPN front-ends (or applications), (b) client configurations, and (c) VPN back-ends. The front-end investigations uncovered several cases of user credential theft and remote code execution. In Abbas et al. (2023), the authors provide a comprehensive survey on VPN technologies, considering security, performance, auditing, and management aspects. Furthermore, they propose a risk assessment methodology specifically designed for VPNs, which they use to evaluate four distinct VPN services. The authors of Ramesh et al. (2022) propose VPNalyzer, a semi-automated tool for VPN solutions, which considers service, security, and privacy aspects. In addition, they utilize the tool to evaluate 80 desktop VPNs of different types (free, paid, and self-hosted). The different tests performed covered, among others, TLS interception, DNSSEC and DoH support, port scan, IP leakage, and geolocation, whereas their main findings include DNS and traffic leaks. Overall, related work on VPN application security does not cover password recovery from system memory, except for (Bui et al., 2019) which partially covers the issue. This highlights that even though it is an existing and known issue, little research has been performed in this direction.

Research gap analysis: While previous studies have extensively examined the security of PMs and VPN applications, especially regarding a wide assortment of vulnerabilities including clipboard attacks, autofill issues, and improper storage of credentials, a significant gap remains: the exfiltration of passwords from the system's volatile memory. Namely, most of the relevant studies, including (Silver et al., 2014; Oesch and Ruoti, 2020; Bui et al., 2019; Gray et al., 2016; Barten, 2019; Lee et al., 2019; Carr and Shahandashti, 2020) concentrated on vulnerabilities such as insecure storage, misconfigurations, or protocol weaknesses. On top of that, existing research (Malliaros et al., 2016; Sabev and Petrov, 2022; Luevanos et al., 2017; Zhao et al., 2013) largely overlooks how memory management techniques, such as zeroization and encryption, impact the persistence of password data in memory. Even more, the only so far existing password recovery tool, dubbed Lazagne (Anon, 2024b), lacks the depth and specificity needed to target a wide range of PM and VPN applications. Recall that Lazagne needs administrative access to extract the relevant credentials. Our research pioneers the investigation of password exfiltration from system memory by examining 12 PMs, 12 PM plugins, 5 browsers, and 21 VPN applications. We assess both system-level memory management flaws and application-specific vulnerabilities that expose sensitive data during runtime. To facilitate this research, we developed Pandora, an open-source tool designed to extract credentials and sensitive data from various PM processes. Besides the wide range of PMs that Pandora supports, it is also noteworthy that it only requires user privileges to extract the credentials. By addressing this significant security gap, our research aims to enhance the security posture of PM and VPN users and organizations.

3. Threat model

Data leakage attacks, which cause sensitive information to be unintentionally exposed to unauthorized parties, directly violate the first property of the Confidentiality, Integrity, and Availability (CIA) triad. For PMs, data leakage can also indirectly affect integrity and availability. If an attacker obtains the PM's master password and subsequently changes it, the legitimate user may be locked out of the PM's vault. Similarly, a user can be locked outside a service account if an attacker changes the user's password. In the case of PMs, where the stored information is considered critical, maintaining all three properties of the CIA triad is paramount. Regarding VPNs, when an attacker obtains the credentials of a VPN connection, they can potentially exploit this issue to gain unauthorized access to internal network resources, modify them and potentially disrupt service. This can lead to consequences such as lateral movement within the corporate environment, Denial of Service (DoS) attacks, or data exfiltration. Overall, within the context of the present work, data leakage refers to cleartext storage of sensitive information in memory, which is commonly known as Common Weakness Enumeration (CWE)-316.

As our experimental evaluation of several renowned PMs and VPNs in Section 4 demonstrates, data leakage remains a significant concern in modern security applications. This section establishes the threat model under which an outsider attacker operates, considering the potential vulnerabilities of both PMs and VPNs in the context of memory-based attacks.

We focus on desktop PMs and VPNs, assuming the targeted machine is within a local network typically protected by a firewall. Within this context, an attacker would need to penetrate the network and gain access to the victim's machine. This can be done in a variety of ways, such as through social engineering, exploiting vulnerabilities, or using brute force attacks. However, these methods are beyond the scope of the current work. Another possibility is for the attacker to mount attacks offline. An illustrative example is when a system is decommissioned without the secure erasure of hard disks. The contents of the RAM could still reside in the disk (for example, through operating system hibernation) and an attacker with physical access to the disk could potentially recover the passwords stored there. Another example is when a user makes a system image for backup purposes; typically, these images contain all the data on the hard drive. Although most imaging software by default excludes hibernation files from such operations, there is no guarantee that these files will be omitted. Considering that system images are stored in external drives or remote file servers, an attacker who gains access to such resources could recover stored passwords when they are not protected.

After gaining initial access, the attacker must obtain a memory dump of the PM or VPN processes. For all PMs except the standalone

version of 1Password, this is straightforward and does not require special privileges. 1Password's processes typically run with a higher integrity level than most other PMs and VPNs, making it more difficult for an attacker to dump their memory and extract sensitive information. Additionally, as outlined in Section 4, in most VPN processes a user only needs medium integrity access to dump the relevant process. Only Windows Client VPN requires *SYSTEM* privileges for the user to dump the relevant process. To elevate privileges and gain access to higher-integrity processes, an attacker may need to bypass User Account Control (UAC). This can be accomplished through techniques such as registry key manipulation, DLL hijacking, or elevated COM interface. A well-utilized DLL hijack-based method is described in hfiref0x (2024). Once the attacker has obtained the memory dumps, they must devise a way to efficiently search each one of these files for sensitive information. This can be done through scripting techniques that implement pattern matching, as detailed in Section 4.6.

Summarizing, considering the key characteristics, goals, and capabilities of potential attackers, the threat model can be described as follows.

- **Goal**: The primary goal of attackers targeting PMs and VPNs is to acquire sensitive user credentials stored in these applications. This information can be used for various malicious purposes, such as identity theft, unauthorized access to accounts, or launching further attacks.
- **Knowledge**: While not necessary in the context of this work, knowledge of vulnerabilities in PMs and VPNs that allow for memory dumps or the extraction of plaintext credentials may be beneficial to attackers. This information can be obtained through research, vulnerability databases, or by exploiting known vulnerabilities.
- **Technical skills**: Attackers need technical skills to recover and analyze memory dumps, identify sensitive information, and potentially exploit any vulnerabilities found. This includes knowledge of reverse engineering, programming, and system administration.
- **Capabilities**: Attackers must have a way to gain access to the target systems running PMs or VPNs, to discarded hard drives, or stored backup files. This could involve exploiting vulnerabilities, social engineering techniques, or physical access to devices. For example, an attacker might trick a victim into downloading and running malicious software, often through a phishing scam. This malware can steal sensitive information stored in a program's memory, such as passwords and other confidential data. This vulnerability arises when programs store such data in an unencrypted form after a user logs in. Moreover, attackers need tools to capture memory dumps of the running PM or VPN processes. This might involve using the operating system's built-in memory forensics tools (as in our case) or custom-built scripts. They must also be able to analyze memory dumps to identify sensitive information, such as plaintext credentials or encryption keys.

4. Experiments

In this section, we present the methodology of our study and our findings. First, we provide details on the testbed and test scenarios used, followed by a discussion of the results of both PMs and VPN clients; finally, we showcase an open source tool we developed, called *Pandora*, for recovering user credentials from memory.

4.1. Testbed and test scenarios

Each PM and VPN application was tested on a separate VMware Workstation Player v17.6 instance running a fresh, up-to-date (Oct. 2024) installation of Windows 10. All VMs were executed on a Windows 10 (build 10.0.19045) laptop with a quad-core processor (8 logical cores enabled via hyperthreading) and 16 GB RAM. Six unique scenarios, S1 through S6, representing different use cases as detailed in Table 2, were developed to test each PM. These scenarios reflect the various operational states a PM may exhibit according to user interactions. Conversely, the VPN applications were assessed solely in scenarios S1 and S6. An observation regarding the behavior of PMs and VPN clients in different OSs is that they leak credentials in both Windows 10 and 11. However, each OS has a dissimilar pattern and number of occurrences for each set of credentials. Notably, the same OS version, regardless of the edition, e.g., Windows 10 Home or Windows 10 Pro, exhibits the same behavior. To retain consistency, keep in mind that all the scenarios were executed with a strict time constraint, *i.e.*, the PM's process(es) were dumped 1 min. after carrying out the respective scenario. Our experimental campaign was undertaken in October 2024, employing the most current versions of standalone, browser and browser plugin PMs, as well as VPN applications that were available at the time.

Referring to the leftmost column of Table 3a, the initial series of experiments tested 12 well-known PM desktop applications (Wired, 2024). In a later phase, as depicted in the leftmost column of Table 4, we repeated all six scenarios using 12 widely-used browser PM plugins (Cybernews, 2024); these tests were conducted on Chrome 129.0.6668.89/.90. By looking at the two tables, it is apparent that half of the PM products are common in both Tables 3 and 4. Moreover, we analyzed 5 well-known browsers that incorporate a PM application; the results are presented in Table 5.

Concerning VPN applications, the first column in Table 6 presents an overview of the 24 products we analyzed, sorted into three main categories: commercial, enterprise, and cloud-based. The table depicts whether a password leak was observed, as well as the number of occurrences for each of the two scenarios used.

4.2. Password managers

Following the execution of each scenario outlined in Table 2, a thorough examination of the resulting dump (.DMP) files was conducted to detect potential exposure of sensitive information in plaintext. The primary focus was on identifying user credentials, including both master passwords and entry passwords associated with PM entries. These entries typically contain the username, password, and URL for various services, such as Facebook. Notably, standalone, browser and browser plugin PMs often spawn multiple processes, resulting in numerous .DMP files when a memory dump is performed. An interesting remark is that master or entry passwords tend to be found in the largest process, although this is not a rule of thumb, as credentials have occasionally been discovered in smaller or multiple files. Due to the arbitrary nature and substantial size (often several megabytes) of these files, manual analysis using a hex editor is impractical. However, this process can be easily automated, as described in Section 4.6.

The outcome of applying all six scenarios to each standalone PM is summarized in Table 3a, where the presence of the ✓symbol signifies password leakage, while the ✗symbol denotes no password leakage. Specifically, the table indicates whether one or more dump files for each PM and scenario contain either the user's master password, entry password, or both. It is important to note that each scenario was executed independently, with the PM application, browser, or browser plugin, being terminated before initiating a new scenario. Furthermore, Table 3a highlights the changes between our current findings and those of our previous study (Chatzoglou et al., 2024b), denoted by cycled ✓or ✗symbols. These differences, observed nearly a year after our initial experiments, underscore the lasting impact of our earlier research. A re-examination of each PM's behavior revealed substantial changes, showing that some of the vendors potentially considered our findings. To further visualize the results, Fig. 1 presents pie charts showing the percentage distribution of master and entry password leaks across different PM applications and scenarios. PMs and scenarios with zero

Table 2
Utilized scenarios for standalone PMs, browser plugin PMs, browser PMs, and VPN applications. The term "relevant processes" in the scenarios refers to either the standalone PM's or the browser's processes. (✓: scenario applicable, ✗: scenario not applicable).

Scenario	Description	Standalone PM/ Browser plugin	Browser PM	VPN
S1	Enter the master password and dump the relevant processes.	✓	✓	✓
S2	Manually lock the PM and dump the relevant processes.	✓	✗	✗
S3	After a certain amount of idle time, the PM is locked automatically; dump the relevant processes.	✓	✗	✗
S4	After creating a new entry password, dump the PM's processes.	✓	✓	✗
S5	While the PM is unlocked, click on a random entry in the corresponding list and dump the relevant processes.	✓	✗	✗
S6	Kill the relevant processes through the task manager, rerun the application without entering the master password, and dump the relevant processes. For VPNs only, manually logout and dump the relevant process without terminating the VPN application.	✓	✓	✓

Table 3
Results for standalone PMs. Master and entry cleartext password leaks are given in white and light-gray background, respectively (✓: password leak, ✗: no password leak, circle: indicates a change compared to our previous study (Chatzoglou et al., 2024b)).

(a) Leaks per type of password per scenario

PM application	Version	S1		S2		S3		S4		S5		S6	
1Password	8.10.46	✓	✗	(✗)	✗	(✗)	✗	(✗)	✓	✓	✗	✗	✗
Bitwarden	2024.9.0	✓	✓	✗	✗	✗	✗	(✗)	✓	✓	✓	✗	✗
Enpass	6.11.3	✗	✗	✗	✗	✗	✗	✗	✓	✗	(✗)	✗	✗
Kaspersky	24.2.0.277	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
KeePass	2.57	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
KeePassXC	2.7.9	✗	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗	✗
Keeper	16.11.3	✓	✓	✗	✗	✓	✓	✓	✓	✓	✓	✗	✗
Nordpass	5.23.10	✓	✗	✗	✗	✗	✗	(✓)	✓	✓	✓	✗	✗
Passwarden	3.3.0	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗
PasswordBoss	5.5.5220.0	✗	✓	✗	✓	✗	✓	✗	✓	✗	✓	✗	✗
RoboForm	9.6.2.2	✓	✓	✓	✗	(✗)	✗	(✗)	(✗)	(✗)	(✗)	✗	✗
StickyPassword	8.8.6.1877	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗

(b) Password instances per scenario

PM application	S1		S2		S3		S4		S5		S6	
1Password	(2)	-	(0)	-	(0)	-	(0)	4	1	-	-	-
Bitwarden	(21)	(18)	-	-	-	-	(0)	(10)	(11)	(18)	-	-
Enpass	-	-	-	-	-	-	-	1	-	(0)	-	-
Kaspersky	-	-	-	-	-	-	-	-	-	-	-	-
KeePass 2	-	-	-	-	-	-	-	-	-	-	-	-
KeePassXC	-	-	-	-	-	-	-	(7)	-	-	-	-
Keeper	(18)	(11)	-	-	(16)	(6)	(5)	(90)	(7)	(12)	-	-
Nordpass	(1)	-	-	-	-	-	(1)	(9)	4	(9)	-	-
Passwarden	(1)	(6)	(4)	(13)	(3)	(22)	(1)	(20)	2	(31)	-	-
PasswordBoss	-	(11)	-	(6)	-	(3)	-	(9)	-	(12)	-	-
RoboForm	1	1	1	-	(0)	-	(0)	(0)	(0)	(0)	-	-
StickyPassword	-	-	-	-	-	-	-	-	-	-	-	-

leaks are excluded for clarity.

With reference to Table 3a, it is evident that only three PMs continue to exhibit no plaintext password exposure across all six scenarios. However, this does not necessarily imply that passwords are absent from memory dumps; they may still be present, but in an obfuscated form that only the PM can recover. On a positive note, re-executing the experiments revealed that no PM leaks occurred in scenario S6, consistent with our previous study (Chatzoglou et al., 2024b). Furthermore, all but one changes observed in the table indicate an improvement in the security posture of the relevant PMs, with 10 instances of password leakage having been patched. The PM with the most patches is RoboForm with 5, followed by 1Password with 3. Notably, only two PMs continue to expose either or both types of passwords in all scenarios except S6, marking an improvement from the four PMs observed in our previous study. For example, RoboForm still leaks both master and entry passwords in S1 and only the master password in S2, but exhibits no leakage in the remaining scenarios. This positive change suggests that the vendor has likely addressed the findings of our earlier study and incorporated security enhancements in the latest PM version.

In addition, only two PMs, Keeper and Passwarden, are found to leak both types of passwords in at least three distinct scenarios. This represents a promising shift, as indicated by the cycled marks in Table 3a, which reveals that Bitwarden and Roboform previously exhibited similar widespread leaks. Notably, Bitwarden no longer leaks the master password in *S4*, while Roboform has eliminated the leakage of both master and entry passwords in *S4* and *S5*. Furthermore, it is

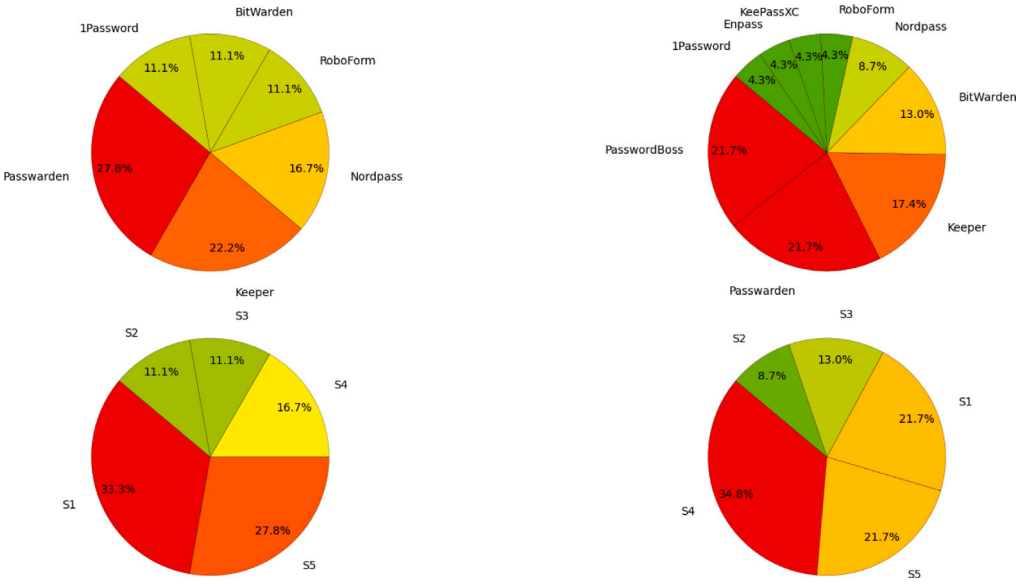


Fig. 1. Distribution of master (left) and entry (right) password leaks across different password managers (top) and scenarios (bottom), with slices arranged from highest to lowest leak percentages, colored from red (highest) to green (lowest). Non-leaking PMs and scenarios do not appear in the diagrams. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 4
Results for PM browser plugins. Master and entry cleartext password leaks are given in white and light-gray background, respectively. (✓: password leak, ✗: no password leak, n/a: either the plugin does not have an automatic lock feature (S3), or the plugin does not offer an "add new entry" feature (S4), or the plugin loads all entries automatically when the browser is started (S5), circle: indicates a change compared to our previous study (Chatzoglou et al., 2024b)).

Browser PM plugin	Version	S1		S2		S3		S4		S5		S6	
1Password	8.10.44.34	✓	✗	✓	✗	✓	✗	✓	✓	✓	✓	✗	✗
Avira	2.21.0.4971	✗	✓	✗	✓	n/a		✗	✓	n/a		✗	✓
Bitdefender	1.3.1	✓	✗	✓	✗	n/a		✗	✓	✗	✓	✗	✗
Bitwarden	2024.10.0	✗	✓	✗	✗	n/a		✗	✗	n/a		✗	✗
Dashlane	6.2440.1	✓	✓	✓	✗	n/a		✓	✓	n/a		✗	✗
Enpass	6.11.0	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
Ironvest	9.9.12	✗	✓	✗	✓	n/a		✗	✗	n/a		✗	✓
Kaspersky	24.2.57.2	✗	✗	✗	✗	✗	✗	n/a		✗	✗	✗	✗
LastPass	4.134.0	✗	✓	✗	✓	n/a		✗	✓	n/a		✗	✓
Norton	8.2.1.388	✗	✓	✗	✓	n/a		✗	✓	n/a		✗	✗
RoboForm	9.6.8.0	✓	✓	✓	✓	n/a		✓	✓	n/a		✓	✓
StickyPassword	8.8.6.1303	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗

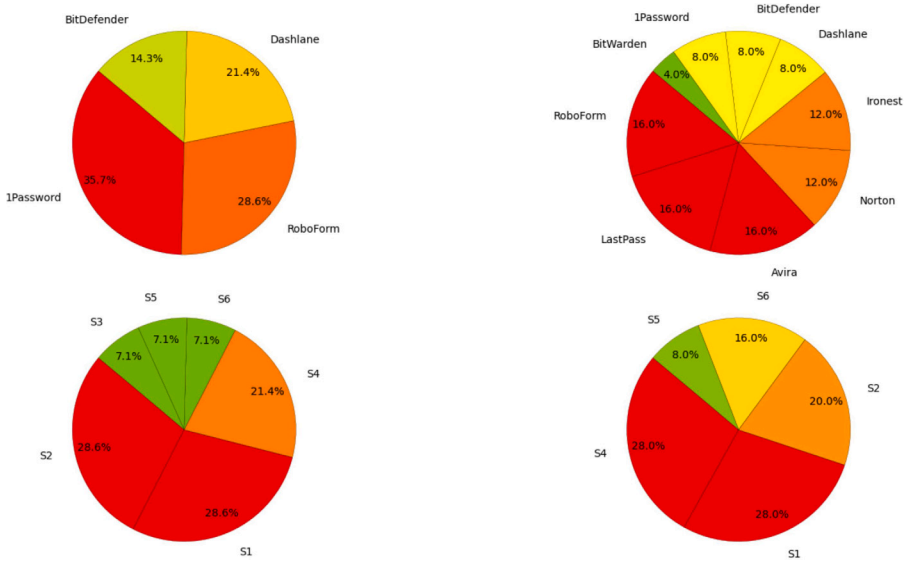


Fig. 2. Distribution of master (left) and entry (right) password leaks across different password manager plugins (top) and scenarios (bottom), with slices arranged from highest to lowest leak percentages, colored from red (highest) to green (lowest). Non-leaking PM plugins and scenarios do not appear in the diagrams. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 5
Results for scenarios applicable to browser PMs; in this case, only entry passwords apply. (✓: password leak, ✗: no password leak).

Browser PM	Version	S1	S4	S6
Brave	1.71.114	✓	✓	✗
Chrome	130.0.6723.59	✓	✓	✗
Firefox	131.0.2	✓	✓	✗
Edge	130.0.2849.46	✓	✓	✓
Opera	114.0.5282.102	✓	✓	✗

observed that half of the PMs still leak the master password in *S1*, which is expected to occur more frequently than the other scenarios in real use. Interestingly, the updated results show no changes in *S1*, suggesting that this particular issue has not been addressed by any of the vulnerable PMs between our two studies.

The situation is not significantly improved for scenarios *S2* to *S5*, where 2, 2, 3, and 5 PMs leak the master password, respectively. However, a reduction in leakage rates is observed in all four scenarios, with 3, 4, 5, and 6 PMs leaking the master password in our previous study (Chatzoglou et al., 2024b). In contrast, an unexpected change is noted in *S4*, where the latest version of Nordpass now leaks the master password, whereas it did not in the earlier version.

Regarding entry passwords, the situation remains largely unchanged from our previous study (Chatzoglou et al., 2024b), with at least two PMs exposing entry passwords throughout scenarios *S1* to *S5*. The worst results are obtained in *S4* and *S5*, where 8 and 5 PMs leak entry passwords, respectively. Although this represents a slight improvement from our previous study, with 9 and 7 PMs leaking entry passwords in *S4* and *S5*, respectively, it still remains a significant issue.

Overall, a total of 41 leaks are observed across all scenarios and PMs' latest versions, consisting of 18 master password leaks and 23 entry password leaks. A review of Table 3a reveals 10 circled ✗symbols and 1 circled ✓symbol, indicating that a substantial number of PM vendors have addressed password leaks in their newer releases. However, the reduction in total number of leaks from 50 to 41 is not a significant improvement, and vendors are urged to take further corrective actions to address these security concerns.

An additional important remark emerged while repeating our experiments with standalone PMs, namely, the frequency of plaintext password appearances in the corresponding memory dump files. This information is summarized in for each scenario and PM. Similar to Table 3a, cycled numbers in the table denote changes in credential occurrences compared to our previous study (Chatzoglou et al., 2024b). It should be clarified that for *S4* and *S5*, the table presents the number of times the newly added (*S4*) or clicked on (*S5*) entry password appears in the respective .DMP file. Upon reviewing , it is evident that the results in both subtables are consistent.

Focusing on , the highest occurrences for entry passwords are observed in *S1*, *S4*, and *S5*, where 3, 6, and 5 PMs, respectively, expose at least 7 instances of the same password. Furthermore, the number of occurrences reaches double digits in *S1* and *S4* for 3 PMs, and in *S5* for 4 PMs. The highest occurrence is 90 in *S4* for Keeper, much higher than the 4 occurrences observed in our previous study. On the other hand, the situation regarding the master password seems better, with the most vulnerable and secure scenarios being *S1* and *S2*, respectively, excluding *S6*. Specifically, in *S1*, 3 PMs expose at least 2 instances of the master password, whereas in *S2*, only Passwarden falls into this category, leaking the master password 4 times.

Generally, the frequency of sensitive data occurrences in a dump file can be highly unpredictable. Specifically, the number of occurrences may vary depending on the time elapsed after the process is spawned or even between executions of the same PM. This volatility may be attributed to inadequate security mechanisms implemented by the PM, such as inefficient garbage collection requests or critical objects overwritten once they are no longer needed. Nevertheless, to maintain consistency, our experiments in both this and our previous study (Chatzoglou et al., 2024b) followed the same execution pattern, dumping the PM's process 1 min after the respective scenario from Table 2 was fulfilled. Although the number of occurrences may not always be identical, repeated leaks of the same sensitive data increase the attacker's chances of identifying the password in the dump file, even through visual inspection. To provide additional clarity, a detailed pseudocode of how this repetitiveness was exploited is presented in Section 4.6 and specifically in Algorithms 2 and 3.

The results for the 12 browser plugins we studied are presented in Table 4 and illustrated in Fig. 2. Notably, only 3 plugins exhibit zero plaintext leaks across all scenarios, with Enpass being a new addition to this category, having previously leaked entry passwords in *S4*. In contrast, 8 plugins leak a password in at least 2 different scenarios, indicating a concerning level of vulnerability. Furthermore, 9 out of 12 plugins, or 75% of the total, leak a password in *S1*, which is considered the most common scenario from a user's perspective. In total, 39 leaks are observed across all scenarios and plugins, consisting of 14 master and 25 entry password leaks. The results also show 5 circled ✗and 21 circled ✓in Table 4, indicating that while some leaks were addressed, a significant number of new vulnerabilities have emerged. For instance, 1Password no longer leaks entry passwords in scenarios *S1* to *S3*, but now leaks the master password in *S1* to *S5*. Additionally, Roboform leaks both master and entry passwords in all applicable scenarios. A notable deterioration compared to our previous study (Chatzoglou et al., 2024b) is that in *S4*, all PM plugins but four now leak entry passwords. Moreover, in *S6*, which is considered the most critical scenario, Ironvest leaks the entry password, and Roboform leaks both types of passwords. On a positive note, Norton no longer leaks any entries in *S6*. Overall, while some PM standalone applications have mitigated existing vulnerabilities, the situation has significantly worsened for browser plugins, with a plethora of new leakages appearing in more recent versions.

To gain a deeper understanding of the results presented in Table 4, several key observations are worth noting. First, some plugins, such as 1Password and Ironvest, require prior interaction between the browser and the plugin itself to leak an entry password. This means that the browser plugin PM dashboard page must be visited before extracting any plaintext information from the relevant process. Second, certain plugins, including Enpass, Kaspersky, and Sticky Password, necessitate the installation of the corresponding desktop PM; as a result, when the desktop PM application is locked, the plugin is also automatically locked. Third, a minority of plugins, such as Roboform, require prior browser interaction with a saved entry URL or the user to click on the plugin's dashboard to load all entries. Fourth, in some cases, such as 1Password, the leak becomes visible after a delay of several seconds, potentially due to the browser taking time to load the respective plugin. Lastly, most plugins do not clear PM-loaded user credentials from the browser's process after a certain time interval has elapsed. Notably, Bitdefender and Bitwarden are the only exceptions to this, clearing the browser's process after 5 and 10 min, respectively.

A comparison of the six common PMs in 3a and 4 reveals that, as expected, there are 2 zero-leak PMs in both their standalone and plugin versions. For the remaining PMs, the situation is generally more or less improved in their plugin version. In addition, these tables highlight that 3 out of 12 standalone PMs (or 25%) and 3 out of 12 browser plugins (or 25%) do not leak any information at all in their process. Notably, there is an overlap between the non-vulnerable standalone PMs (Kaspersky, KeePass, and StickyPassword) and the non-vulnerable PM browser plugins (Enpass, Kaspersky, and StickyPassword).

Regarding the results presented in Table 5, which depict the password leakage of browser PMs, we note that scenarios *S2*, *S3*, and *S5* are not applicable. This is because browsers do not possess a lock feature, and all entries are automatically loaded when the PM is open in the browser. Focusing on *S1*, *S4*, and *S6*, the following key points can be

Table 6
Credential leak and occurrences of VPN applications tested. (✓: credential leak, ✗: no credential leak, *: uses a unique 16-digit token instead of a password).

VPN client	Version	S1 Leak	S1 Occ.	S6 Leak	S6 Occ.
Commercial					
CyberGhost	8.4.9.14404	✗	–	✗	–
ExpressVPN	12.88.0(36)	✗	–	✗	–
Hide.me VPN	4.2.0	✓	1	✓	1
Hotspot Shield	12.9.3	✗	–	✗	–
IPVanish	4.2.7.6	✓	1	✓	2
Mullvad VPN	2024.5.0	✓*	18	✓*	28
NordVPN	7.28.4.0	✗	–	✗	–
Norton Ultra VPN	24.8.9372	✗	–	✗	–
PIA	3.6.1	✗	–	✗	–
PrivateVPN	3.0.11	✗	–	✗	–
ProtonVPN	3.3.2	✗	–	✗	–
PureVPN	13.9.1.0	✗	–	✗	–
Surfshark	5.10.1	✓	1	✓	1
TunnelBear VPN	4.9.6	✗	–	✗	–
Total VPN	6.0.1530	✗	–	✗	–
Windscribe	2.11.11	✗	–	✗	–
Enterprise					
Cisco AnyConnect	5.1.4.74	✗	–	✗	–
Fortinet FortiClient	7.4.0.1658	✓	4	✗	–
OpenVPN Connect	3.5.0	✓	3	✓	2
Palo Alto GlobalProtect	6.3	✗	–	✗	–
Windows VPN Client	Windows 10 Pro 22H2 19045.5011	✓	1	✗	–

observed. First, all browser PMs leak entry passwords in cleartext format within their process in both *S1* and *S4*; additionally, in *S6*, only one browser PM leaks this sensitive information. It should be noted that for *S6*, the browser must be simply open, without visiting the PM browser page. In the latter case, most browsers offer protections to prevent easy access to the PM browser page, making it difficult to open the browser and visit the PM page from command-line terminals. For instance, using the command "`start chrome chrome://password-manager/passwords`" in the command prompt (cmd) does not open the PM of the Chrome browser, but instead, only an empty tab. However, there are alternative methods to open a browser's PM from a terminal, such as executing a VBScript, although this approach is not particularly Operationally Secure (OPSEC) friendly.

4.3. Virtual private networks

Regarding the experiments on VPN providers, we analyzed each product based on the first (*S1*) and sixth (*S6*) scenarios of Table 2. The reason for this is that the remaining scenarios, namely *S2* to *S5*, do not apply to VPNs. For instance, a VPN cannot be locked or gain an entry. Due to the functionality of VPN applications, a different approach was required in *S6* scenario, where the behavior involves logging out manually and dumping the relevant process without terminating the related application. In examining secret information in cleartext within the process of a VPN, we focused on user credentials, specifically either a username/email address or a password. The results of each examined scenario are summarized in Table 6 for each VPN product, where the ✓ and ✗ symbols indicate a credential leak and no credential leak, respectively. With respect to Table 6, we categorized VPN providers based on their target group, *i.e.*, commercial or enterprise. Enterprise VPNs typically require hardware to operate in conjunction with a client VPN application. We managed to gain access to 5 such VPN clients.

In addition to the categories presented in the table, there exists another type of VPN, namely cloud VPNs, which are not explicitly listed. Cloud VPN providers often rely on third-party VPN clients, such as the Windows VPN Client or OpenVPN Connect, to facilitate communication with their cloud services. Consequently, if a user utilizes a cloud VPN service in conjunction with a VPN client that leaks credentials in plaintext within memory, they will also be exposed to the same vulnerabilities. Our study included three cloud-based VPNs: Azure Cloud VPN (utilizing the Windows VPN Client), AWS Cloud VPN (using OpenVPN Connect), and Google Cloud VPN (also using OpenVPN Connect).

Overall, out of 21 VPN clients, 7 or 33% reveal plaintext credentials in memory in at least one scenario. Broken down to each scenario, 7 VPN clients show credential leakage in *S1* and 5 in *S6*. Notably, each VPN client that leaks credentials in *S6* is also leaking credentials in *S1*. Regarding commercial VPN clients, our analysis reveals that out of the 16 examined, only 4 keep their credentials in plaintext within their application's memory, accounting for 25% of the total. Notably, within this subset of VPN clients, 3 of them display 1 or 2 instances of plaintext in their memory for each scenario, whereas Mullvad VPN exhibits a higher number of occurrences with 18 in *S1* and 28 in *S6*. Note that the results of Mullvad are denoted with an asterisk in Table 6. This is because user authentication is not achieved through user credentials, *i.e.*, username and password, but with a unique 16-digit token generated upon the user's registration with the Mullvad VPN client. However, as mentioned above, this token is still visible multiple times in cleartext in the respective dump files. Furthermore, our examination of the 5 enterprise VPN clients reveals that 3 of them are vulnerable to credential leakage; all of them are vulnerable in *S1* whereas only 1 in *S6*. In terms of frequency, the occurrences of plaintext credentials in both scenarios range from 1 to 4.

4.4. Password managers vs VPN clients

Noticeably and comparing Tables 3a, 4, 6, and 5 we can observe that PMs, in all their forms, are more susceptible to leaking sensitive information in memory. Specifically, 9 of 12 standalone PMs, equating to 75%, and an equal proportion of browser plugins, leak information in at least one scenario. Even worse, this figure on browser PMs reaches 100% as all 5 browser PMs leak passwords in at least 2 scenarios. Regarding VPNs, 7 out of 21 clients, or 33%, exhibit information leakage in at least one scenario. Consequently, the likelihood of data leakage in any of the three PM forms is over twice that of a VPN.

4.5. Discussion

Abiding by a Coordinated Vulnerability Disclosure (CVD) process, the results were promptly shared with the majority of the affected PM and VPN vendors, along with the corresponding exploits. Four vendors in total acknowledged the issue; two PM and one VPN vendors assigned a Common Vulnerabilities and Exposures (CVE) ID, specifically CVE-2023-23349, CVE-2024-9203, and CVE-2024-50570. The fourth one,

which is a VPN vendor, plans to issue a CVE ID after implementing stable fixes. The remaining vendors also acknowledged the issue but provided justifications in two main directions: (a) they did not consider the vulnerability significant, or (b) they were uncertain about how to effectively address it.

The first approach taken by some vendors implies that they are already aware of the problem, but they tend to downplay its significance. Specifically, their primary argument is that carrying out such an attack would require a compromised system and a malicious actor with escalated privileges. On the one hand, this assertion is half true; as it is evident from our results in Section 4.2 there do exist PM and VPN products that do not leak secret information in plaintext. Notably, there are instances of standalone PMs (three cases), browser plugins (three cases), and VPN clients (fourteen cases) that do not exhibit this vulnerability.

On the other hand, as detailed in Section 3, all PMs except one do not require elevated privileges for dumping their processes; the same applies to all VPN clients except one. This means that the attacker does not need to bypass UAC, which is typically a challenging task. Some vendors argue that keylogger malware can achieve the same goal without collecting and inspecting a PM memory dump file. However, there are several counterarguments to this claim. Firstly, a keylogger needs to operate stealthily and continuously, waiting for the victim to type a password. In contrast, an attacker can save the relevant .DMP file and analyze it offline, without the need for real-time monitoring. Additionally, if the PM divulges the master password, the attacker gains instant access to every entry stored in the PM's vault, making it a more severe vulnerability than a keylogger.

The second line of arguments, although less common, suggests that some PM and VPN vendors are uncertain about how to effectively address the issue. Specifically, they are concerned that applying some form of obfuscation to the passwords loaded in memory may provide a false sense of security. This approach is essentially a form of security by obscurity, which is only effective as long as the secret obfuscation pattern remains confidential.

On the other hand, unauthorized access to the PM can lead to multiple breaches, equaling the number of different entry passwords stored in the PM. Furthermore, leaking VPN credentials can enable attackers to gain unauthorized network access and subsequently move laterally. In the absence of a stronger defense, obfuscation appears to be the only viable solution, at least for the time being. To enhance the robustness of the obfuscation procedure, a variable and frequently changing pattern should be employed, ideally in conjunction with other techniques, such as the system-protected process security model, also known as Protected Process Light (PPL) technology (Anon, 2022b). For more information on this issue, the reader is referred to Section 5.

4.6. Red teaming and exploits

To propel research and advance red teaming techniques, as well as encourage ethical vulnerability disclosure, we have developed an open-source tool called *Pandora* (Chatzoglou et al., 2024a). This tool is designed to facilitate the extraction of user credentials, including both usernames and passwords, that may be found in cleartext in the processes initiated by various well-known PMs. Publicly available via a GitHub repository (Anon, 2024f), *Pandora* is compatible with standalone PMs on MS Windows 10, given that in browser and browser plugin PMs the number of password appearances is not stable. As of the latest update, it supports 4 browser-based PMs and 14 different PMs, in 18 different configurations. This is possible by extracting credentials from either the desktop version or the browser extension variant of a single PM. The main requirement for using *Pandora* is that the targeted PM must be active and in an unlocked state. For additional and up-to-date information on the tool, the interested reader is referred to Anon (2024f).

To get a better understanding of the inner workings of the tool, we present three key algorithms: 1, 2, and 3. Algorithm 1 outlines a generic exploit for pattern-based gathering of credentials using pseudocode. This algorithm works by applying a search pattern (line 3) to the process memory dump of a PM after opening the respective .DMP file. The search pattern is based on the common technique used by anti-malware services, where a file is flagged as potentially malicious if a specific sequence of hex bytes (signature) is present.

Depending on the case, to identify potential patterns we configured each PM with different credentials, typically, master and entry passwords along with their matching usernames. We then used a hex editor to manually investigate if each dump file contained one or more of these entries. If an entry was found, we extracted the preceding and succeeding bytes as possible search patterns. These patterns were then tested across different PM states to determine if they remained consistent. If a consistent pattern was identified, it could be used to create an exploit for the specific PM. Algorithm 1 demonstrates how to gather the next or previous n bytes, convert them to UTF-8, and reveal the password and username in plaintext (lines 5–21). The value of n is determined empirically and may vary depending on the PM; for this algorithm, n is set to 300 bytes (line 12). Note that in each repetition until reaching the end of the dump file, the pattern data are saved to a file (line 17).

Algorithm 1: getCreds

Input: *app.dmp*
Output: *status*

```
1  Function getCreds():
2      Open file "app.dmp" in binary mode ;
3      searchPattern ← [0x80, 0x00, 0x04, ..., 0x00] ;
4      foundData ← empty list ;
5      while not end of file do
6          c ← read a byte from file ;
7          if c = next element in searchPattern then
8              Add c to foundData ;
9              if length of foundData = length of searchPattern then
10                 extractedData ← empty list ;
11                 dataCount ← 0 ;
12                 while dataCount < 300 and not end of file do
13                     extractedData ← read a byte and add to
                         extractedData ;
14                     Increment dataCount ;
15                 utf8ExtractedData ← convert extractedData to UTF-8
                     string ;
16                 Print "Pattern Data: " concatenated with
                     utf8ExtractedData ;
17                 saveFile(utf8ExtractedData) ;
18                 Clear foundData ;
19         else
20             Clear foundData ;
21     Close app.dmp ;
22     return 0 ;
```

Algorithms 2 and 3 present a generic exploit in pseudocode to analyze the repetitiveness of a password within a process, that is, to determine how many times a password is present. Due to its time-consuming nature, it is recommended to use Algorithm 2 in conjunction with a pattern-based exploit to reduce the output pattern data. For instance, relying solely on repetitiveness may require up to 20 min to identify a password. In our case, Algorithm 2 takes as input a .txt file containing search patterns, such as the one populated by Algorithm 1 in line 17. The data following these search patterns could be either passwords or non-password (noise) data. To decrease the number of the returned data that we need to analyze for discovering the actual passwords, we first search for password occurrences in the specific PM

manually; this results to the occurrences shown in . The calculated number of occurrences is then used as variable X in line 15. This way, an attacker can calculate manually the occurrences for each PM once and then use this number across different dump files from the specific PM, automating the password recovery process.

In more detail, the pseudocode of Algorithm 2 reads the contents of the pattern file (line 9), separates each line into a sequence (line 12), and converts this sequence to bytes (line 13). It then calls Algorithm 3 to search the contents of the file and identify the number of occurrences of each sequence (line 14). If the number of occurrences is the same as the one identified manually (line 15), the sequence probably contains a password; note that in the end one of the identified sequences will contain a password. This sequence is then printed to the user as well as saved into a file for further analysis (lines 16–17).

Algorithm 2: getCredsRepetitiveness

```
   Input: filename.txt
   Output: status
1  Function getCredsRepetitiveness(filename):
2      Open file "filename.txt" for reading ;
3      if "filename.txt" is not open then
4          Print "Error opening the sequences file." ;
5          return 1 ;
6      if filename is not open then
7          Print "Error opening the data file." ;
8          return 1 ;
9      fileData ← read all contents of filename into a list of bytes ;
10     sequencesFile ← opened "filename.txt" ;
11     while not end of sequencesFile do
12         line ← read next line from sequencesFile ;
13         sequence ← convert line to list of bytes ;
14         occurrences ← countOccurrences(fileData, sequence) ;
15         if occurrences = X then
16             Print "Sequence: " concatenated with line and " found X
                 time." ;
17             saveFile(line + ε\nε) ;
18     Close filename.txt ;
19     return 0 ;
```

Algorithm 3: countOccurrences

```
   Input: data, sequence
   Output: count
1  Function countOccurrences(data, sequence):
2      count ← 0 ;
3      it ← iterator at start of data ;
4      while it ≠ end of data do
5          it ← search(it, end of data, sequence) ;
6          if it ≠ end of data then
7              Increment count ;
8              Move it one step forward ;
9      return count ;
```

5. Best practices

The current section briefly describes best practices that should be employed for sensitive information management by a PM or VPN application, to mitigate associated risks. As discussed in Section 4, the success of the attack relies on the fact that sensitive information is stored in plaintext format by these applications. Oddly enough, current state-of-the-art standards (Anon, 2022a; Grassi et al., 2020; Souppaya et al., 2016) do not recognize this issue as a critical aspect of their information risk management frameworks. For example, the ISO/IEC:27001 standard (Anon, 2022a) underlines that "*security measures shall be implemented when personnel are working remotely to protect information accessed, processed or stored outside the organization premises*". Similarly, a NIST special publication (Grassi et al., 2020) provides a catalog of security and privacy controls to protect sensitive information from unauthorized access and disclosure. While these standards promote the implementation of security measures to safeguard sensitive data, they overlook memory leakage issues, focusing on other essential security aspects such as access control, system and communications protection, and media protection.

In contrast to existing standards, there are guidelines (Chapman, 2017; OWASP, 2024b,a) that provide recommendations for such applications to properly handle private information. The Open Worldwide Application Security Project (OWASP) guidelines emphasize the importance of employing cryptographic primitives (OWASP, 2024b,a) to protect private information. Encryption and hashing algorithms are considered essential tools for PM developers to safeguard private information, although key management remains a crucial aspect of the process. Additionally, OWASP suggests using obfuscation techniques to hide essential information when loaded in memory, provided that the obfuscation pattern remains secret. Furthermore, the pattern should be frequently changed, which entails management operations, while there is no silver bullet that can ensure key management security horizontally. OWASP also recommends that private data should be stored only when necessary during the application life cycle (OWASP, 2024a) and remain in RAM for the minimum possible time (OWASP, 2024c). This can be achieved by spawning garbage collection requests after critical operations or overwriting the content of critical objects once they are no longer needed. Moreover, employing memory zeroing techniques before deallocation can help ensure that sensitive information is not inadvertently left in memory after use.

During the application development stage, another commonly used technique is to eliminate variable copies (Chapman, 2017). Modern programming languages often create multiple copies of the same information in the memory stack through various operations on variables. For instance, each variable assignment can create a copy of the same information. This might be the case with the analyzed PMs, as shown in , where both the master and entry passwords were identified multiple times in the respective memory dumps. This makes it easier for an attacker to monitor the repeating data and reveal recurring private information with associated byte patterns. Some PMs, such as Keeper, offer an option to wipe secret data after the PM is automatically locked. However, this option is opt-in and disabled by default, which contradicts the principle of establishing secure defaults. Moreover, only a few PMs support Two-Factor Authentication (2FA), which is a positive feature. Unfortunately, in some cases, the 2FA token created during user sign-in with the PM also leaks into the memory.

Remaining to software development practices, developers should also employ secure memory management techniques to prevent sensitive data from being swapped to disk. One approach is to use functions that allocate memory in a way that prevents it from being swapped out. For example, developers can use mmap with MAP_LOCKED flags on Unix-like systems (Anon, 2024e) or VirtualAlloc with the MEM_RESERVE and MEM_COMMIT flags on Windows (Anon, 2024d) to ensure that sensitive data remains in physical memory only. This reduces the risk of exposure through page swapping. Moreover, locking memory regions that contain sensitive information can prevent attackers from accessing this data through techniques such as memory dumps or process introspection. By taking these measures, developers can further protect sensitive data and reduce the risk of unauthorized access.

As previously discussed in Sections 3 and 4.5, developers can leverage the UAC mandatory access control enforcement feature to limit the ability of malicious code to execute with administrator privileges. UAC requires applications to prompt the end-user for approval, known as the ''elevation prompt'' when requesting administrator access. This ensures that processes execute with the user's token, restricting their capabilities to those of the user. In this context, when an MS Windows thread attempts to access a protected object, the operating system

conducts a security check, verifying the thread's token to determine its permissions. Additionally, vendors can utilize Protected anti-malware services, also known as PPL (Anon, 2022b), to run specially-signed programs in a mode resistant to tampering and termination, even by administrative users. However, running a PM with administrative rights to enable PPL may introduce risks, such as privilege escalation attacks through DLL sideloading, as seen in CVE-2023-48861.

A hardware-based approach to securing sensitive data involves the use of secure enclaves. Secure enclaves, such as Intel SGX and ARM TrustZone, utilize hardware features to create a protected area in memory that is only accessible to the application that created it. This shielded environment ensures that sensitive data processed within the enclave is safeguarded from being read or modified by external processes, making it significantly more difficult for attackers to exploit vulnerabilities related to memory access (Torrey and Sherman, 2016). Notably, secure enclaves are employed by various prominent entities, including Apple products (Anon, 2024a), Microsoft's Azure confidential computing (Anon, 2024c), and secure vehicle communication systems (Lima et al., 2016). However, the underlying system must meet specific hardware requirements to support secure enclaves, which may limit their adoption. A closely related concept is trusted execution environments (TEEs) (Sabt et al., 2015), which define a secure area of the main system processor and allow isolated execution of applications. The main difference between a security enclave and a TEE is that the former is hardware-based, whereas a TEE can be implemented either in hardware or software.

6. Conclusions & future work

Despite the widespread use of password credentials for user authentication in numerous applications, questions remain about their secure management and usage. While PMs offer a convenient solution to the problem of managing multiple account credentials and creating strong passwords, it is essential to evaluate their trustworthiness. Can PMs truly be relied upon to store sensitive information securely, or do they introduce new vulnerabilities? Moreover, even when using strong passwords from a PM that does not leak sensitive information in local applications, such as VPN clients, what is the level of assurance that credentials will not leak to unauthorized parties?

This paper presents a comprehensive security analysis of popular standalone, browser, and browser plugin PMs, as well as several commercial and enterprise VPN clients. The analysis is conducted under the assumption that an adversary has gained access to the local system and can dump the respective processes. The findings reveal that, across six typical end-user scenarios, a significant percentage of PMs and VPN clients store sensitive information in plaintext in system memory while in use. Specifically, the results show that out of a total of 29 PM applications, 75% of desktop PMs, 100% of browser PMs, and 75% of browser plugin PMs store passwords in plaintext in system memory. Additionally, 33% of the 21 examined VPN clients leaked sensitive user credentials. The main issue with these security-sensitive applications is that they create patterns in memory that can be easily identified by an adversary. Furthermore, this sensitive information remains in memory for an extended period, even after it is no longer needed, and is often stored in multiple copies, making it easier for an attacker to access.

Although this study focuses solely on two types of applications, namely, PMs and VPN clients, we believe that this kind of vulnerability may impact any client-side application that relies on user authentication via security credentials. Consequently, we anticipate that it may raise concerns for other similar and widely used software, such as Remote Desktop Protocol (RDP) applications and SSH clients. In this regard, a potential direction for future work is to extend this research to mobile platforms and various other popular applications of different kinds, including financial/banking, team collaboration, smart home, and more.

CRediT authorship contribution statement

Efstratios Chatzoglou: Writing – review & editing, Writing – original draft, Software, Resources, Methodology, Investigation, Conceptualization. **Vyron Kampourakis:** Writing – review & editing, Writing – original draft, Software, Resources, Methodology, Investigation, Conceptualization. **Zisis Tsiatsikas:** Writing – review & editing, Writing – original draft, Validation, Software, Resources, Methodology, Investigation, Conceptualization. **Georgios Karopoulos:** Writing – review & editing, Writing – original draft, Validation. **Georgios Kambourakis:** Writing – review & editing, Writing – original draft, Validation, Supervision, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the Research Council of Norway through the SFI Norwegian Centre for Cybersecurity in Critical Sectors (NORCICS) project no. 310105 and by the European Union through the Horizon 2020 project PERSEUS (Grant No. 101034240).

Data availability

We have shared the link to our source code in the references.

References

Abbas, H., Emmanuel, N., Amjad, M.F., Yaqoob, T., Atiquzzaman, M., Iqbal, Z., Shafqat, N., Shahid, W.B., Tanveer, A., Ashfaq, U., 2023. Security assessment and evaluation of VPNs: A comprehensive survey. ACM Comput. Surv. 55 (13s), http://dx.doi.org/10.1145/3579162.

Anon, 2022a. Information security, cybersecurity and privacy protection — information security management systems — Requirements. URL https://www.iso.org/standard/27001. (Last visited 31/10/2024).

Anon, 2022b. Protecting anti-malware services. URL https://learn.microsoft.com/en-us/windows/win32/services/protecting-anti-malware-services-. (Last visited 31/10/2024).

Anon, 2024. Top 10 best VPNs. URL https://www.techradar.com/vpn/best-vpn. (Last visited 31/10/2024).

Anon, 2024a. Apple platform security: Secure enclave. URL https://support.apple.com/en-my/guide/security/sec59b0b31ff/web. (Last visited 31/10/2024).

Anon, 2024b. The LaZagne project. URL https://github.com/AlessandroZ/LaZagne. (Last visited 31/10/2024).

Anon, 2024c. Microsoft: Azure confidential computing. URL https://azure.microsoft.com/en-us/solutions/confidential-compute. (Last visited 31/10/2024).

Anon, 2024d. Microsoft: VirtualAlloc function (memoryapi.h). URL https://learn.microsoft.com/en-us/windows/win32/api/memoryapi/nf-memoryapi-virtualalloc. (Last visited 31/10/2024).

Anon, 2024e. mmap: Linux manual page. URL https://man7.org/linux/man-pages/man2/mmap.2.html. (Last visited 31/10/2024).

Anon, 2024f. Pandora. URL https://github.com/efchatz/pandora. (Last visited 31/10/2024).

Apostolopoulos, D., Marinakis, G., Ntantogian, C., Xenakis, C., 2013. Discovering authentication credentials in volatile memory of android mobile devices. In: Conference on E-Business, E-Services and E-Society. pp. 178–185.

Barten, D., 2019. Client-side attacks on the LastPass browser extension. URL https://api.semanticscholar.org/CorpusID:153315475.

Bui, T., Rao, S., Antikainen, M., Aura, T., 2019. Client-side vulnerabilities in commercial VPNs. In: Askarov, A., Hansen, R.R., Rafnsson, W. (Eds.), Secure IT Systems. Springer International Publishing, Cham, pp. 103–119.

Carr, M., Shahandashti, S.F., 2020. Revisiting security vulnerabilities in commercial password managers. In: Hölbl, M., Rannenberg, K., Welzer, T. (Eds.), ICT Systems Security and Privacy Protection. Springer International Publishing, Cham, pp. 265–279.

Chapman, R., 2017. Sanitizing sensitive data: How to get it right (or at least less wrong...). In: Reliable Software Technologies – Ada-Europe 2017. pp. 37–52.

Chatzoglou, E., Kampourakis, V., Tsiatsikas, Z., 2024a. Pandora: Exploit password management software to obtain credential from memory. Presented at Black Hat Europe Arsenal, London, UK, https://www.blackhat.com/eu-24/arsenal/schedule/index.html#pandora-exploit-password-management-software-to-obtain-credential-from-memory-42259.

Chatzoglou, E., Kampourakis, V., Tsiatsikas, Z., Karopoulos, G., Kambourakis, G., 2024b. Keep your memory dump shut: Unveiling data leaks in password managers. In: Pitropakis, N., Katsikas, S., Furnell, S., Markantonakis, K. (Eds.), ICT Systems Security and Privacy Protection. Springer Nature Switzerland, Cham, pp. 61–75.

Cybernews, 2024. Chrome password managers: Best extensions in 2023. https://cybernews.com/best-password-managers/password-managers-for-chrome. (Last visited 31/10/2024).

Fagan, M., Albayram, Y., Khan, M.M.H., Buck, R., 2017. An investigation into users' considerations towards using password managers. Human-centric Comput. Inf. Sci. 7 (1), 12.

Fung, B., Sands, G., 2021. Former SolarWinds CEO blames intern for 'solarwinds123' password leak. URL https://edition.cnn.com/2021/02/26/politics/solarwinds123-password-intern/index.html. (Last visited 31/10/2024).

Gangwal, A., Singh, S., Srivastava, A., 2023. AutoSpill: credential leakage from mobile password managers. In: Proceedings of the Thirteenth ACM Conference on Data and Application Security and Privacy. pp. 39–47.

Grassi, P.A., Fenton, J.L., Newton, E.M., Perlner, R., Regenscheid, A., Burr, W.E., Richer, J.P., Lefkovitz, N., Danker, J.M., Choong, Y.-Y., et al., 2020. Digital identity guidelines: Authentication and lifecycle management [includes updates as of 03-02-2020]. NIST Special Publication.

Gray, J., Franqueira, V.N.L., Yu, Y., 2016. Forensically-sound analysis of security risks of using local password managers. In: 2016 IEEE 24th International Requirements Engineering Conference Workshops. REW, pp. 114–121.

hfiref0x, 2024. Defeating windows user account control by abusing built-in windows AutoElevate backdoor. URL https://github.com/hfiref0x/UACME#readme. (Last visited 31/10/2024).

Ikram, M., Vallina-Rodriguez, N., Seneviratne, S., Kaafar, M.A., Paxson, V., 2016. An analysis of the privacy and security risks of android vpn permission-enabled apps. In: Proceedings of the 2016 Internet Measurement Conference. IMC '16, Association for Computing Machinery, New York, NY, USA, pp. 349–364. http://dx.doi.org/10.1145/2987443.2987471.

International Organization for Standardization, 2022. ISO/IEC 27001:2022 information technology — Security techniques — information security management systems — Requirements. https://www.iso.org/standard/54534.html. (Accessed: 15 october 2024).

Kotadia, M., 2004. Gates predicts death of the password. URL https://www.cnet.com/news/privacy/gates-predicts-death-of-the-password. (Last visited 31/10/2024).

Lee, J., Chen, A., Wallach, D.S., 2019. Total recall: Persistence of passwords in Android. In: Proceedings 2019 Network and Distributed System Security Symposium. URL https://api.semanticscholar.org/CorpusID:142503488.

Li, Z., He, W., Akhawe, D., Song, D., 2014. The {emperor's} new password manager: Security analysis of web-based password managers. In: 23rd USENIX Security Symposium (USENIX Security 14). pp. 465–479.

Lima, A., Rocha, F., Völp, M., Esteves-Veríssimo, P., 2016. Towards safe and secure autonomous and cooperative vehicle ecosystems. In: Proceedings of the 2nd ACM Workshop on Cyber-Physical Systems Security and Privacy. pp. 59–70.

Luevanos, C., Elizarraras, J., Hirschi, K., Yeh, J.-h., 2017. Analysis on the security and use of password managers. In: 2017 18th International Conference on Parallel and Distributed Computing, Applications and Technologies. PDCAT, pp. 17–24.

Malliaros, S., Ntantogian, C., Xenakis, C., 2016. Protecting sensitive information in the volatile memory from disclosure attacks. In: 2016 11th International Conference on Availability, Reliability and Security. ARES, pp. 687–693.

Oesch, S., Ruoti, S., 2020. That was then, this is now: A security evaluation of password generation, storage, and autofill in browser-based password managers. In: Proceedings of the 29th USENIX Conference on Security Symposium. pp. 2165–2182..

OWASP, 2024a. Cryptographic storage cheat sheet. URL https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html. (Last visited 31/10/2024).

OWASP, 2024b. Secure product design cheat sheet. URL https://cheatsheetseries.owasp.org/cheatsheets/Secure_Product_Design_Cheat_Sheet.html. (Last visited 31/10/2024).

OWASP, 2024c. Testing memory for sensitive data. https://mas.owasp.org/MASTG/tests/android/MASVS-STORAGE/MASTG-TEST-0011/. (Last visited 31/10/2024).

PASCU, L., 2017. 29 vulnerabilities found in top-rated password managers for android. URL https://www.bitdefender.com/blog/hotforsecurity/29-vulnerabilities-found-in-top-rated-password-managers-for-android/. (Last visited 31/10/2024).

Pearman, S., Zhang, S.A., Bauer, L., Christin, N., Cranor, L.F., 2019. Why people (don't) use password managers effectively. In: SOUPS 2019. pp. 319–338.

Perta, V.C., Barbera, M.V., Tyson, G., Haddadi, H., Mei, A., 2015. A glance through the VPN looking glass: IPv6 leakage and DNS hijacking in commercial VPN clients. Proc. Privacy Enhanc. Technol. 2015 (1), 77–91. http://dx.doi.org/10.1515/popets-2015-0006, Received 11/22/2014; revised 2/16/2015; accepted 2/17/2015.

Ramesh, R., Evdokimov, L., Xue, D., Ensafi, R., 2022. VPNalyzer: Systematic Investigation of the VPN Ecosystem. In: Proceedings 2022 Network and Distributed System Security Symposium. URL https://api.semanticscholar.org/CorpusID:248223212.

Sabev, P., Petrov, M., 2022. Android password managers and vault applications: Data storage security issues identification. J. Inf. Secur. Appl. 67 (C).

Sabt, M., Achemlal, M., Bouabdallah, A., 2015. Trusted execution environment: What it is, and what it is not. In: 2015 IEEE Trustcom/BigDataSE/ISPA, vol. 1. pp. 57–64. http://dx.doi.org/10.1109/Trustcom.2015.357.

Silver, D., Jana, S., Boneh, D., Chen, E., Jackson, C., 2014. Password managers: Attacks and defenses. In: 23rd USENIX Security Symposium (USENIX Security 14). pp. 449–464.

Smiliotopoulos, C., Kambourakis, G., Kolias, C., 2024. Detecting lateral movement: A systematic survey. Heliyon e26317. http://dx.doi.org/10.1016/j.heliyon.2024.e26317, URL https://www.sciencedirect.com/science/article/pii/S240584402402348X.

Souppaya, M., Scarfone, K., et al., 2016. Guide to enterprise telework, remote access, and bring your own device (BYOD) security. NIST Special Publ. 800, 46.

sysdig, 2023. KeePass CVE-2023-32784: detection of processes memory dump. https://sysdig.com/blog/keepass-cve-2023-32784-detection/. (Last visited 31/10/2024).

Torrey, J.I., Sherman, B.M., 2016. Enclaves for operating system protection. In: 2016 Annual IEEE Systems Conference. SysCon, pp. 1–5. http://dx.doi.org/10.1109/SYSCON.2016.7490626.

Turton, W., Mehrotra, K., 2021. Bad password gave colonial pipeline hackers access. URL https://www.governing.com/security/bad-password-gave-colonial-pipeline-hackers-access. (Last visited 31/10/2024).

Wilson, J., McLuskie, D., Bayne, E., 2020. Investigation into the security and privacy of iOS VPN applications. In: Proceedings of the 15th International Conference on Availability, Reliability and Security. ARES '20, Association for Computing Machinery, New York, NY, USA, pp. 1–9. http://dx.doi.org/10.1145/3407023.3407029.

Wired, 2024. The best password managers to secure your digital life. https://www.wired.com/story/best-password-managers/. (Last visited 31/10/2024).

Wu, K.L., Hue, M.H., Poon, N.M., Leung, K.M., Po, W.Y., Wong, K.T., Hui, S.H., Chau, S.Y., 2023. Back to school: On the (In)Security of academic VPNs. In: 32nd USENIX Security Symposium (USENIX Security 23). USENIX Association, Anaheim, CA, pp. 5737–5754, URL https://www.usenix.org/conference/usenixsecurity23/presentation/wu-ka-lok.

Zhang, Q., Li, J., Zhang, Y., Wang, H., Gu, D., 2018. Oh-Pwn-VPN! security analysis of OpenVPN-based android apps. In: Capkun, S., Chow, S.S.M. (Eds.), Cryptology and Network Security. Springer International Publishing, Cham, pp. 373–389.

Zhao, R., Yue, C., Sun, K., 2013. Vulnerability and risk analysis of two commercial browser and cloud based password managers. Acad. Sci. Eng. Sci. J. 1, 183–197.